NTNU
Kunnskap for en bedre verden

Leksjon 10 - Produktforvaltning

Forvaltning av produktene del I – Planlegging og utvikling

Knut Arne Strand, Institutt for datateknologi og informatikk (IDI), NTNU.

Lærestoffet er utviklet for faget Produktutvikling, marked og forvaltning av programvareprodukter

1. Forvaltning av produktene del I – Planlegging og utvikling

Resymé: I denne leksjonen ser vi nærmere på rammeverket som vi introduserte i forrige leksjon. Dette heter «Software Product Management Reference Framework» eller «The software product management body of knowledge» og i denne leksjonen fokuserer vi på andre av tre deler. Hovedfokuset er planlegging og utvikling av nye programvarebaserte produkter og tjenester, med spesiell vekt på utarbeidelse av krav til slike løsninger.

Innhold

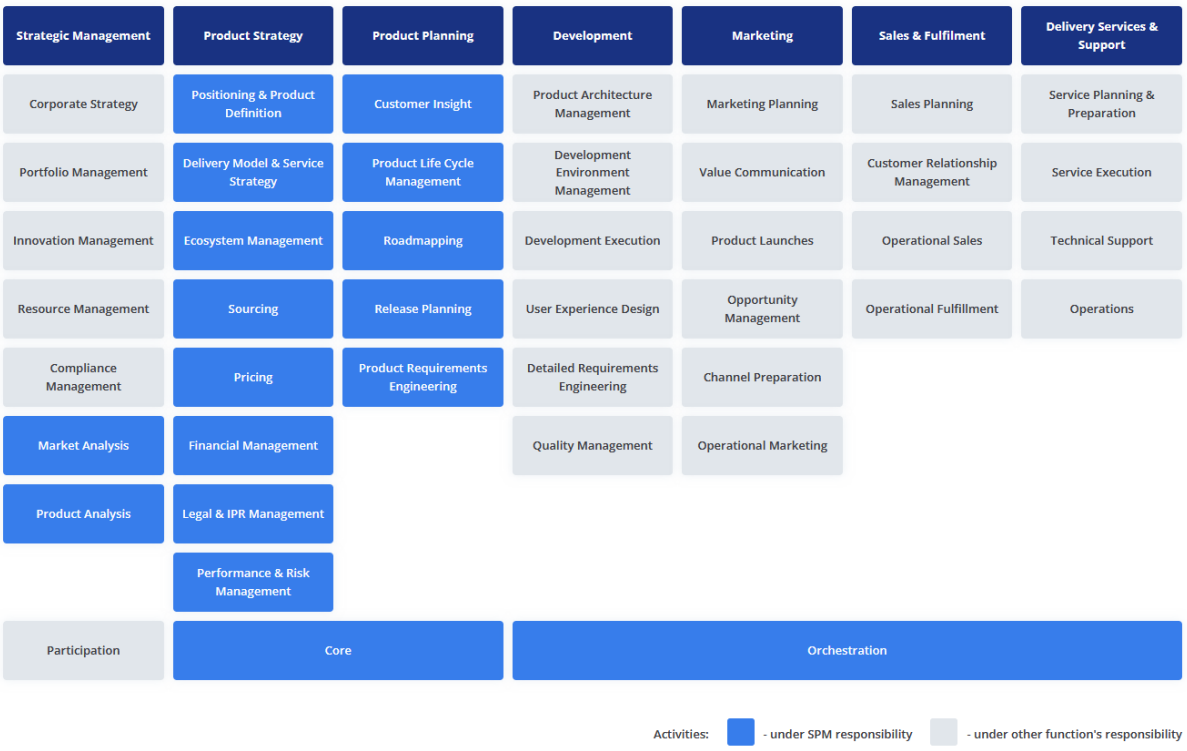
1. FORVALTNING AV PRODUKTENE DEL I – PLANLEGGING OG UTVIKLING 1

1.1. INNLEDNING 2
1.2. PRODUKTPLANLEGGING 2
1.2.1. Kundeinnsikt 3
1.2.2. Styring og ledelse gjennom produktenes livssyklus 4
1.2.3. Veibeskrivelse for produktene 5
1.2.4. Planlegging av nye utgivelser 7
1.2.5. Utvikling av krav til produktene 8
1.3. PRODUKTUTVIKLING 12
1.3.1. Forvaltning av programvarearkitektur 12
1.3.2. Forvaltning av utviklingsmiljø 13
1.3.3. Utvikling 14
1.3.4. UX Design 15
1.3.5. Detaljert kravhåndtering 15
1.3.6. Kvalitetsledelse 16
1.1. REFERANSELISTE 17

Opphavsrett: Forfatter(e) og Stiftelsen TISIP

1.1. Innledning

Dette er tredje leksjon innen produktforvaltning som henter mye av sitt innhold fra kapittel 4 (s. 137 til 210) og første del av kapittel 6 (s. 243 til 256) i læreboken. Hovedfokuset for denne leksjonen er planlegging og utvikling av nye programvarebaserte produkter og tjenester, med spesiell fokus på utarbeidelse av krav til slike løsninger.



Figur 1 - The software product management body of knowledge

Figur 1 viser en oversikt over rammeverket som vi introduserte tidligere og vi skal nå se nærmere på produktplanlegging (eng. Product Planning) og utvikling (eng. Development). Som det framgår av figur 1 (kolonne 3) så regnes produktplanlegging som kjerneaktiviteter for en produktsjef, mens utvikling (kolonne 4) inneholder aktiviteter hvor produktsjefen orkestrerer. Dette kan vi tolke som at produktsjefen kan være med å styre og påvirke mye av arbeidet uten at vedkommende nødvendigvis gjør selve jobben. Når vi i de kommende avsnitt gjennomgår deler av dette rammeverket så henter vi mye av innholdet fra kildene [1] [2], [3], [4]

1.2. Produktplanlegging

Produktplanlegging (eng. Product Planning) går ut på å beskrive hva slags programvareprodukter man trenger, hvordan de skal være sammensatt og hva de skal bestå av, når de skal utvikles, hvordan utviklingen skal foregå, når de skal være klar for markedet og hvilket marked de tilhører. Sammen med produktstrategi så utgjør produktplanlegging kjerneaktivitetene innen produktforvaltning. Produktsjefen definerer her

Opphavsrett: Knut A Strand og Stiftelsen TISIP

programvareproduktet, setter mål og rammer for utviklingen av produktet, sørger for at man kan oppnå lønnsomhet og skape et fornuftig forretningsområde rundt produktet og sørger for god støtte og engasjement fra produktets interessenter. Produktsjefen må ta styringen og lederansvaret for utviklingen av produktet og de beslutninger som må tas i den sammenheng. Avhengig av om produktet skal selges i det kommersielle markedet (altså at man jobber for en programvareleverandør) eller om produktet er et internt informasjonssystem, må man tilpasse hvor mye planleggingen skal omhandle markedsføring og salgsaktiviteter

En produktsjef bruker mye tid på å planlegge forskjellige ting rundt produktene. Dette kan kanskje oppfattes som uproduktivt for de som står utenfor, men det er svært viktig å bruke tid på planarbeidet når man representerer et produkt eller en hel produktfamilie. Planleggingsarbeidet foregår i mange sammenhenger og det kan omfatte markedsplaner, salgsplaner, budsjettering og planlegging av ressursbruk i tilknytning til produktene. Grunnleggende spørsmål som en produktsjef jobber med i planleggingsfasen er å avgjøre hvilke ressurser (mennesker, penger, utstyr) som må dedikeres til produktene på kort sikt (daglig drift), på mellomlang sikt (neste versjon) og på lang sikt (neste generasjon). Beslutningene må baseres på markedssituasjonen og forventede inntektsprognoser, hvor produktet befinner seg i livssyklusen, og hvilke avhengigheter som eksisterer i forhold til andre produkter som ligger i eller utenfor produktfamilien. Produktplanlegging tilhøre kjerneaktivitetene innen produktforvaltning og vi skal nå se nærmere på de fem hovedaktivitetene som *The software product management body of knowledge* (se figur 1) trekker fram i denne sammenheng.

1.2.1. Kundeinnsikt

På engelsk benyttes begrepet Customer Insight om det som vi på norsk kan kalle kundeinnsikt. Kundeinnsikt handler om å forstå kundenes behov, ønsker og atferd for å kunne utvikle, forvalte og markedsføre programvareprodukter som tilfredsstiller disse kravene mest mulig effektivt.

For å oppnå best mulig kundeinnsikt kan vi f.eks. gjennomføre markedsundersøkelser for å samle kvantitative data om markedstrender, kundeadferd og preferanser. Dette kan inkludere spørreundersøkelser, analyser av brukeratferd eller konkurrentanalyser. Videre kan vi benytte kvalitative forskningsmetoder som dybdeintervjuer, fokusgrupper og brukertesting for å få innsikt i kundenes tanker, følelser og holdninger i forhold til et bestemt programvareprodukt.

En identifikasjon og inndeling av produktets målgruppe i ulike kundesegmenter, basert på felles behov og egenskaper kan også hjelpe oss med å tilpasse produkter og markedsføringen av disse til spesifikke grupper. I denne sammenheng kan f.eks. opprettelse av bruker-personas, som er fiktive representasjoner av ideelle kunder, hjelpe oss med å forstå kundenes mål, utfordringer og behov på en dypere måte. En annen tilnærming kan være å kartlegge hele kundereiser, fra bevissthet om produktet oppstår til kjøp gjennomføres og produktet tas i bruk. En slik kartlegging kan hjelpe oss med å identifisere hvor og hvordan kunder interagerer med programvareproduktet. Aktiv innsamling av tilbakemeldinger fra kunder gjennom ulike kanaler som kundestøtte eller brukerforum kan også gi oss innsikt i hva som fungerer bra og hva som kan forbedres.

Vi trenger å forstå hvordan kundene bruker produktet, hvilke problemer de støter på i den daglige bruken og hvilke funksjoner i produktet de setter mest pris på. God kundeinnsikt er avgjørende for å utvikle programvareprodukter som faktisk løser kundenes problemer og

Opphavsrett: Knut A Strand og Stiftelsen TISIP

oppfyller deres forventninger. Denne innsikten danner grunnlaget for å etablere en effektiv produktstrategi, samt produktutvikling og markedsføring som treffer best mulig i forhold til målgruppen. For mer informasjon om kundeinnsikt, se side 137-140 i læreboken [1, s. 137–140]

1.2.2. Styring og ledelse gjennom produktenes livssyklus

På engelsk benyttes begrepet Product Life Cycle Management om det som vi her velger å kalle styring og ledelse gjennom produktenes livssyklus. En produktsjef er ansvarlig for produktet langs hele dets livssyklus. Produktenes livssyklus kan deles inn i forskjellige faser som har sine spesielle egenskaper og fokusområder og hvor forskjellige aktører i tillegg til produktsjefen, er involvert. I de første fasene står investeringer sentralt siden disse er nødvendige for å utvikle produktet. Etter hvert som produktet blir modent og etablert i markedet, og dersom man lykkes med dette, vil produktet fungere som en melkeku. Det vil si at produktet genererer betydelige inntekter samtidig som investeringsnivået er lavt. Det overskuddet man oppnår i denne fasen kan typisk brukes til å finansiere andre lovende produkter i porteføljen. I følge læreboken [1, s. 196] kan produktenes livssyklus deles inn i følgende 6 faser:

1. Konseptualiseringsfasen – I denne fase skapes og utvikles produktet. Dette er en innovativ fase hvor investeringer gjøres og i denne fasen må produktsjefen samarbeide med de som driver FoU-aktiviteter, de som jobber mot markedet for å avdekke behovene, samt eventuelle organer som regulerer markedet.
2. Introduksjon til markedet – I denne fasen skal produktet lanseres i markedet og man skal gjøre investeringer som skal gi avkastning i form av høyere markedsandeler. Produktsjefen må typisk jobbe tett mot markedsavdelingen og salgsavdelingen i denne fasen.
3. Vekstfasen – Dette er fasen hvor markedsandelene vokser og hvor det investeres i at produktet skal få utvidet funksjonalitet. Utvikling, markedsføring og salg er sentrale oppgaver i denne fasen og produktsjefen kan typisk og til en viss grad, involveres i alle disse oppgavene.
4. Modenhetsfasen – I denne fasen kan man typisk fokusere på revitalisering rundt produktet, f.eks. ved å utvide tjenestetilbudet. I denne fasen kan det være fokus på tjenestetilbudet og kundestøtten rundt produktet, men salgsfunksjonen kan også være sentral.
5. Tilbakegangsfasen – Før eller siden når alle produkter en fase hvor tilbakegangen er et faktum. I denne fasen har man typisk fokus på å beholde kundene og det kan være markeds-, salgs- og/eller kundestøtteavdelingen som bidrar her.
6. Utfasing – Til slutt vil produktet fases helt ut og det er gjerne fokus på å få de siste kundene over på alternative produkter eller tjenester. Bedriften ønsker her å bli kvitt kostnadene i forbindelse med forvaltning av det utdøende produktet og det er gjerne markedsavdelingen som er aktiv mot kundene i denne fasen.

Produktsjefen bør ha en god forståelse for de ulike ovennevnte fasene, slik at vedkommende er i stand til å utvikle forskjellige strategier med tilhørende aktiviteter som bidrar til å støtte produktene i de forskjellige fasene på en optimal måte. For å lykkes med dette må produktsjefen samarbeide tett med andre funksjoner som f.eks. utvikling, salg, markedsføring,

Opphavsrett: Knut A Strand og Stiftelsen TISIP

tjenesteutvikling og kundestøtte. Man kan si at dette dreier seg om produktrelatert kunnskapsforvaltning gjennom produktenes livssyklus. Altså det at produktsjefen må forsikre seg om at den kunnskapen som kreves for å opprettholde et levedyktig produkt faktisk er tilgjengelig og vil fortsette å være det gjennom produktets levetid. Det å gjennomføre undersøkelser og målinger som avslører om produktet fungerer i forhold til forventningene er avgjørende for å kunne sette inn korrigerende tiltak der det er nødvendig. En produktsjef må derfor både overvåke og analysere hvor godt produktet fungerer og dette inkluderer både lønnsomhet, vurdering av faktiske mot planlagt inntekter, kundetilfredshet og markedsandeler.

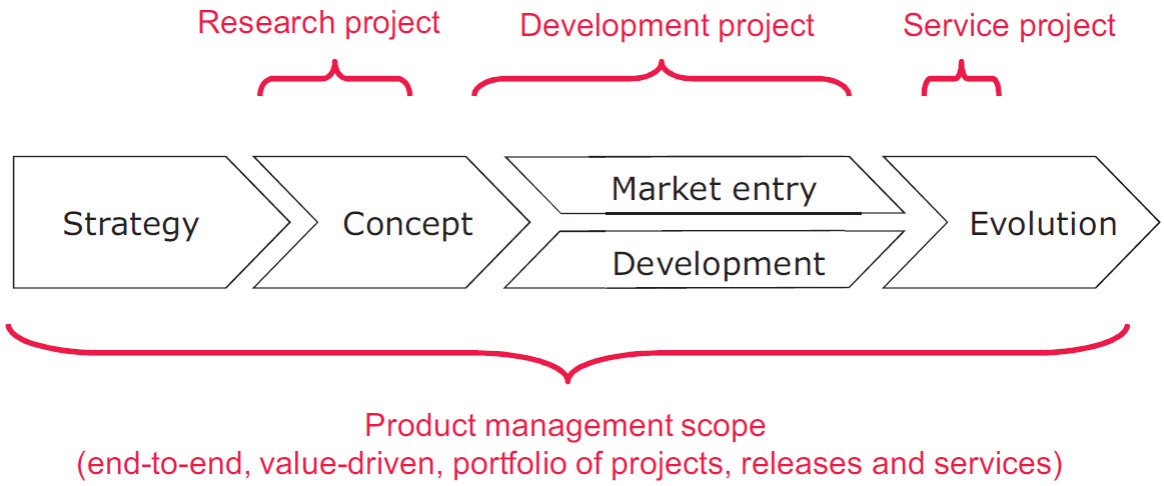


Fig. 2 – Produktforvaltning gjennom hele produktet sin livssyklus.
Hentet fra artikkelen Software Product Management–An Industry Evaluation [5].

Figur 2 gir en oversikt over en typisk livssyklus for et produkt hvor vi ser at flere prosjekter inngår i end-til-ende framstillingen av produktets livssyklus. Figuren fremhever forskjellene mellom det å lede et prosjekt og det å forvalte et produkt. Et prosjekt er et midlertidig/tidsbegrenset arbeid for å lage noe, gjerne et produkt. Forskjellen mellom prosjektledelse og produktledelse er akkurat dette fokuset på å levere et bestemt produkt i tide, i henhold til budsjett og med riktig sammensetning og kvalitet som prosjektlederen må fokuserer på, mens produktsjefen ser på produktets totale markedssuksess og utviklingen av produktet over tid med kommende versjoner og tilhørende tjenester.

For mer informasjon om «Styring og ledelse gjennom produktenes livssyklus» se læreboken [1, s. 195–204].

1.2.3. Veibeskrivelse for produktene

Veibeskrivelse for produktene (eng. Roadmap) handler om å overføre produktets strategi til en rekke med kommende utgivelser av produktet. Denne rekken av utgivelser skal tilfredsstille selskapets forretningsmessige mål, innenfor en strategisk tidsramme, som typisk kan være mellom ett og fem år. Jeg kjenner ikke til noen norsk ord som er dekkende for dette så jeg velger å benytte «Roadmap» og «Roadmapping» i noen sammenhenger, i fortsettelsen. Et «Roadmap» kan typisk bestå av følgende:

- En tidsplan som viser når forskjellige hendelser skal skje.

Opphavsrett: Knut A Strand og Stiftelsen TISIP

- Hvilke versjoner av programvaren som skal komme på gitte tidspunkt.
- Hva de forskjellige versjonen omhandler (Hovedfunksjoner)
- Hvilke markeder de forskjellige versjonene henvender seg til
- Eventuelle avhengigheter til andre produkter
- Eventuelle teknologiske påvirkninger

Tidspunktet for når utgivelsene skal komme er ofte styrt av overordnede føringer og gjeldende policy i forhold til utgivelser. Det vil si en frekvens hvor produktsjefen definerer at større eller mindre utgivelser skal komme. Det som er input og førende når vi skal lage et «Roadmap» er: 1) hvilke visjoner og hvilken strategi vi har for produktet og hvordan dette skal realiseres, 2) hvilke planer som eksisterer for nye utgivelser, 3) resultater fra utvikling av krav til produktene og 4) andre «Roadmaps» som kan tilhøre andre produkter firmaet selv er ansvarlig for, eller som eksterne partnere er ansvarlig for. Det kan f.eks. eksistere andre teknologiske «Roadmaps» som et «Roadmap» for et produkt må tilpasses.

Veibeskrivelser for produktene «Product Roadmaps» lages både for interne og eksterne interessenter. En intern veibeskrivelse beskriver typisk hva som skal inngå i spesifikke produktlanseringer. Dette gir grunnlag for salgsprognoser, budsjettering og etablering av utviklingsprosjekter som har mål om å ferdigstille spesifikke produktlanseringer. Eksterne veibeskrivelser brukes typisk til å etablere en kommunikasjonskanal mellom kundene og selskapet. På denne måten tjener veibeskrivelsen flere hensikter: (1) den blir et hjelpemiddel til å kommunisere strategien rundt produktene, (2) den blir et middel som også blir utgangspunkt for å samle tilbakemeldinger, og (3) den bidrar til å bygge tillit og engasjement ved at man formidler selskapets langsiktige planer for investering i produktet. Den planleggingen av nye utgivelser som kan leses ut av veibeskrivelsene kan i sin tur bli input til en mer detaljert planlegging av nye utgivelser. Det er dermed et samspill mellom utarbeidelse av veibeskrivelser for produktene og planlegging av nye utgivelser.

Selve arbeidet med å utvikle veibeskrivelser for produktene kalles «Roadmapping» på engelsk. Dette er en aktivitet som er utberedt og det finnes flere forskningsartikler om dette temaet. Følgende definisjon er hentet fra en av disse artiklene:

“Roadmapping is a popular metaphor for planning and portraying the use of scientific and technological resources, elements and their structural relationships over a period of time. The process of roadmapping identifies, evaluates and selects strategic alternatives that can be used to achieve desired objectives […], and the resulting roadmaps summarise and communicate the results of key business decisions […]. Product roadmapping is a ”disciplined, focused, multiyear approach to product planning”, with the roadmap’s implementability viewed as important as its strategic value […]. “ [6, s. 2].

Fordeler og ulemper

Mange av fordelene ved «roadmapping» er ikke utelukkende i selve «roadmappet», men prosessen som resulterer i «roadmappet». I forbindelse med utarbeidelsen av «roadmappet» møtes ulike aktører som jobber innen forskjellige avdelinger med forskjellige arbeidsoppgaver, og det er i denne prosessen informasjon, meninger og ulike perspektiv drøftes, og mye av nytten fremkommer.

Det finnes eksempler på programvareprodusenter som hevder at det å benytte «roadmapping» styrker sammenhengen mellom forretningsmessige beslutninger og utvikling av

Opphavsrett: Knut A Strand og Stiftelsen TISIP

kravspesifikasjoner. Samtidig må man også være oppmerksom på at «roadmaps» fort blir utdatert og at det er en krevende jobb å holde de oppdatert. Et utdatert «roadmap» er totalt ubrukelig og kan fort føre til misforståelser som fører til at beslutninger tas på feil grunnlag.

Videre kan et «roadmap» stå i fare for å bli et ormebol av blokker, piler, notasjoner og skriblerier. Dette kan være fullstendig forståelig og danne et nøyaktig bilde for de som har produsert det, men det kan virke uforståelig og uoversiktlig for resten av bedriften som skal lese og forstå det. Når dette skjer sier man gjerne at man ikke har klart å temme «roadmappet» eller at det har kommet ut av kontroll.

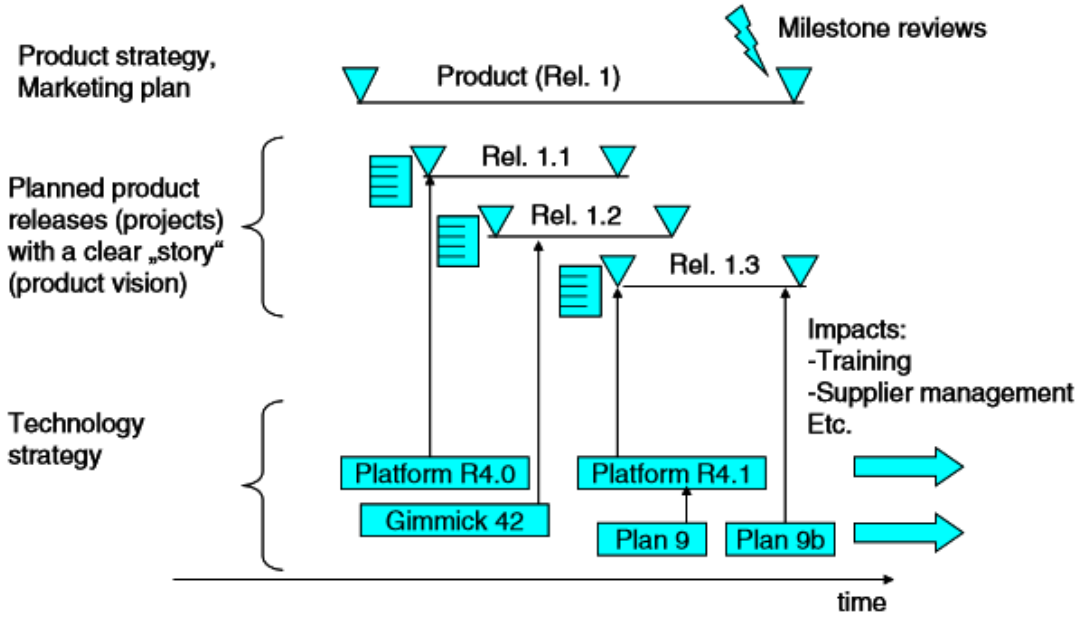


Fig. 3 – En veibeskrivelse for produktutgivelser. Hentet fra [7].

I figur 3 vises forholdet mellom produktstrategien og veibeskrivelsen. Produktvisjonen legger overordnede føringer, men den må i nest omgang samstemmes med utviklingen i den teknologien som man er avhengig av (se nederst i figuren). Med utgangspunkt i hva man ønsker skal være med i utgivelsene (funksjonelle prioritering) og hvilke teknologiske løsninger som er tilgjengelig, så planlegges utgivelsene. I forbindelse med hver enkelt produktutgivelse, utarbeides det materiale som kan bli brukt til markedsføring salgsaktiviteter, etc. Mer om planlegging av nye utgivelser i neste avsnitt.

For mer informasjon om «Veibeskrivelse for produktene» se læreboken [1, s. 184–195].

1.2.4. Planlegging av nye utgivelser

Planlegging av nye utgivelser (eng. Release Planning) handler om å bestemme hva slags funksjonalitet som skal inngå som en del av en ny utgivelse. Dersom man velger å utvikle et produkt gjennom flere iterasjoner (funksjonelle utvidelser slippes med jevne mellomrom) så vil man tidligere begynne å tjene penger på investeringen. En slik inkrementell utvikling krever at man prioriterer godt og «riktig» i forhold til hva som skal være med på hvilke

Opphavsrett: Knut A Strand og Stiftelsen TISIP

utgivelser. Man må forsikre seg om at det er god kvalitet på de utgivelsene som slippes. Samtidig må man ta hensyn til ting som at løsningene skal være nyttig for kunden, at man opprettholder konkurransekraften, at kapasiteten er god nok, at man legger opp til en utgivelsesplan som gjør at man blir forutsigbar, at de forskjellige interessentene blir hensyntatt og at avhengigheter mellom forskjellige krav blir ivaretatt.
På grunn av forretningsmessige, ledelsesmessige og markedsføringsmessige grunner vil programvareprodukter finnes i forskjellige utgaver. Den langsiktige planen med utgivelser og tilhørende innhold (funksjonalitet) planlegges og dokumenteres i form av en veibeskrivelse for produktene (eng. roadmap). Avhengig av hva slags endringer som inngår og hvor stort omfang vi snakker om så kan en utgivelse kalles en hoved-utgivelse (eng. major), en liten utgivelse (eng. minor), en oppdatering (eng. update) eller en oppdateringspakke (eng. service pack). Hovedutgivelser er ofte og av markedsmessige årsaker kalt nye versjoner.

Planlegging av nye utgivelser går som nevnt på å definere innholdet i en kommende versjon av produktet på tilstrekkelig detaljeringsnivå. Samtidig skal produktsjefen eller den som står for planleggingen forsøke å maksimere produktets verdi, både i forhold til den nærmeste utgivelsen og i forhold til hele produktets livssyklus. På den ene siden må funksjonaliteten som inngår tilfredsstille forretningsmessige mål og kundens reelle behov, samtidig som man oppnår fordeler i forhold til konkurrentene og samtidig som man henger med teknologisk. På den andre side må man ta hensyn til utviklerorganisasjonen, de planer som er lagt og de ressurser man faktisk rår over i forhold til utvikling av produktet. Det kan også være at inngåtte kundeavtaler, salgs- og markedsføringsaktiviteter, planlagte opptredener på messer, etc., kan påvirke hva som skal inngå i en kommende utgivelse.
Planlegging av nye utgivelser er derfor i stor grad forhandlinger og prioriteringer mellom konkurrerende behov, siden forskjellige interessenter kan ha sterke føringer i forhold til hva som skal inngå og siden man derfor må løse konflikter mellom forskjellige interessenter. Hvilke beslutninger man velger å treffe når man planlegger en ny utgivelse bør også være basert på overordnede og strategiske føringer eller gjeldende retningslinjer. Disse kan typisk inneholde i hvilken grad selskapet skal være reaktiv i forhold til sine markeder ved å prioritere kundebehov eller om man skal ha en proaktiv strategi som f.eks. sørger for at produktene ligger langt framme rent teknologisk.
Resultatene fra dette planleggings- og prioriteringsarbeidet blir etter hvert dokumentert i form av en plan for kommende utgivelser og denne må interessentene bare slutte seg til. Hvordan man organiserer dette arbeidet kan imidlertid være avhengig av hvilken utviklingsmodell som benyttes. I smidige utviklingsprosjekter (f.eks. SCRUM) så må produktsjefen (SCRUM Product Owner) være delaktig og involvere seg i forhold til hva som skal inngå i den kommende løsningen, gjennom hele utviklingsprosjektet. Kravene til løsningene blir ikke beskrevet i detalj på forhånd, men de blir løpende utviklet og beskrevet i mer detalj, etter hvert som prosjektet skrider fram.

For mer informasjon om «Planlegging av nye utgivelser» se læreboken [1, s. 169–184].

1.2.5. Utvikling av krav til produktene

Utvikling av krav til produktene (eng. Products Requirements Engineering) kalles også markedsdrevet utvikling av krav til produktene (eng. market-driven requirements engineering). Dette handler om å samle inn krav til produktene fra forskjellige interessenter, som har forskjellige behov og forventninger, for så å la disse kravene styre utviklingen av programvareproduktet.

Opphavsrett: Knut A Strand og Stiftelsen TISIP

Produktsjefen utfører typisk prioriteringer for å redusere mengden av krav som skal tilhøre neste versjon og for å sikre at utviklingsprosjektet blir gjennomførbart og har et hensiktsmessig fokus. Kravene til systemet bearbeides og beskrives som konkrete produktegenskaper (både funksjonelle og teknologiske egenskaper) som skal bli en del av de kommende versjonene.
Utvikling av krav til et programvareprodukt, sett fra en produktsjef sitt ståsted, inneholder klassiske oppgaver som å få fram kravene (eng. elicitation), å spesifisere kravene i detalj, å validere kravene og å sette utviklingen av kravene ut i live. Dette må avstemmes med markedsmessige behov hvor brukere, kunder, konkurrenter og leverandører kommer med føringer. Krav kan deles inn i følgende tre kategorier:

- Krav fra kunden – Dette er krav som uttrykker den enkelte kunde sine behov og det er vanligvis salgsavdelingen, som en del av kundeoppfølgingsprosessen (Customer Relationship Management – CRM) som har ansvaret for å samle inn og formidle disse kravene videre.
- Markedsmessige krav – Dette er krav som ikke kommer fra enkeltstående kunder, men snarere fra en rekke kunder som kan ha like behov, men hvor det også kan være en viss grad av variasjon mellom behovene. Dette markedsmessige fokuset påvirker mye av arbeidet med utvikling av krav til produktene og da spesielt dette med å få fram kravene (eng. elicitation) og å ta beslutninger rundt hva som skal inkluderes i kommende versjoner av produktene. Det er produktsjefen sitt ansvar å utvikle krav til produktene og dette arbeidet er tett koplet til oppgaver som å etablere en visjon for produktene, å definere en strategi for produktene, å utarbeide veibeskrivelser for produktene (eng. roadmaps), FoU aktiviteter og planlegging av nye utgivelser. Produktkrav føre ofte til krav og føringer for utviklingsprosjektene, men det finnes også produktkrav som adresserer andre områder. Dette kan være nye prismodeller, at forskjellige produkter settes sammen til en produktfamilie, at leveransemodellene endres eller at supportapparatet og støttefunksjonene rundt produktene endres og forbedres.
- Prosjektsmessige krav er de krav til produktene som skal implementeres og realiseres i en bestemt versjon eller et bestemt prosjekt og disse kravene blir typisk realisert ved programmering. Prosjektsmessige krav kan også adressere interne behov som oppstår i løpet av et utviklingsprosjekt og dette er normalt prosjektlederen og utviklingsprosjektet sitt ansvar.

En produktleverandør trenger en egen prosess for å håndtere kravene til produktene på en systematisk måte. Dette blir en syklisk prosess som omhandler å få fram kravene (eng. elicitation), å gjennomføre en innledende evaluering, å analysere og spesifisere kravene i detalj, å velge ut hva som skal realiseres i hvilke versjoner, samt å validere de valg som er gjort. Her følger en gjennomgang av disse oppgavene:

- Å få fram kravene (eng. elicitation) er systematisk bruk av metoder som intervjuer, arbeidsmøter, observasjon av brukerne, kreativ utarbeidelse av krav, forskjellige undersøkelser og analyse av systemene og hvordan de brukes. Hensikten med dette systematiske arbeidet er å forstå fagområdet (domenet) som applikasjonen skal fungerer innenfor, samt å identifisere interessentene og avdekke hvilke mål og forventninger de har. En viktig kilde til å avdekke krav kan også være innad i bedriften hvor f.eks. FoU-aktiviteter kan stå sentralt. Når kravene er hentet fram skal de dokumenteres og dette kan gjøres på flere måter. F.eks. som tekstlige beskrivelser (f.eks. brukerhistorier (eng. user stories) eller som modeller (f.eks. brukstilfellemodeller (eng. use cases)).

Opphavsrett: Knut A Strand og Stiftelsen TISIP

- Innledende evaluering (eng. triage) går ut på å gjennomføre en første analyse for å avgjøre om hvert enkelt krav er godt nok beskrevet og om det skal være en del av produktet. Dette bidrar til å avgjøre om man trenger å gjennomføre flere detaljerte analyser eller ikke. I denne fasen kan man også sortere kravene slik at de som hører sammen, håndteres i felleskap siden man på denne måten kan øke produktiviteten. Produktsjefen vil typisk vedlikeholde en liste med et betydelig antall krav som er relevant for produktet, men som ikke er implementert og som gjerne må vurderes, evalueres og prioriteres flere ganger.
- Å analysere og spesifisere kravene i detalj går ut på å avstamme informasjon som er hentet fram i tilknytning til kravene mot aktuelle løsningsforslag. Dette innebærer å utarbeide detaljerte spesifikasjoner, utarbeide aktuelle modeller (f.eks. brukstilfellemodeller) og å prototype løsningsforslag som i neste omgang fungerer som input til beslutningsprosesser. Når kravene er spesifisert på tilstrekkelig detaljeringsnivå skal man også utarbeide estimater for implementeringen. Altså hvor mye tid som vil medgå, hvilke kostnader som vil påløpe og hvilke ressurser man trenger for å realisere kravene.
- Å velge ut hva som skal realiseres i hvilke versjoner handler om å beslutte hvilke krav som skal implementeres i et bestemt utviklingsprosjekt. Dette er altså en del av prioriteringsarbeidet som inngår i arbeidet med å planlegge en ny utgivelse.
- Validering av krav (eng. requirements validation) handler om å bruk av metoder som systematiske gjennomganger for å sikre at den foreslåtte løsningen (inklusive krav som skal realiseres) tar tak i de utfordringer man virkelig har og er akseptable for interessentene. Altså å avgjøre om man har prioritert riktig. Dersom man har lyktes med utviklingen av krav til produktene skal man kunne ende opp med en spesifikasjon som dokumenterer hvilke avtaler som er inngått mellom utviklerne og produktets interessenter slik at de har en felles forståelse av hva som vil bli levert.

Når man jobber med utvikling av krav til produktene kan det være hensiktsmessig å kategorisere kravene i forhold til hvor de befinner seg i kravhåndteringen sin livssyklus. Dette kan gjøres på flere måter og det vil variere fra produktleverandør til produktleverandør. Her er imidlertid noen eksempler som illustrerer dette

- Nye krav – Dette er krav som akkurat er introdusert.
- Godkjente krav – Dette er krav som er godkjent og som derfor kan analyseres videre.
- Spesifiserte krav – Dette er krav som har blitt analysert med tanke på hvordan de skal realiseres og hvor mye ressurser som trengs (estimat).
- Avvist – Dette er krav som avvises. Dette kan skyldes at det allerede er implementert, at det finnes tilsvarende krav (duplikater) eller at kravet ligger på utsiden av hva systemet faktisk skal løse.
- Valgt til implementering – Dette kravet er valgt ut for å være med på en kommende versjon og det har fått tildelt en prioritet.
- Implementert – Dette kravet er ferdig utviklet og inngår som en del av produktet.
- Ferdig testet – Testing av dette kravet er fullført og man har dermed forsikret seg om at kvaliteten er tilstrekkelig.
- Realisert – Alle aktiviteter i forbindelse med utgivelsen av produktet er utført for dette kravet. Kravet inngår altså i en versjon av produktet som er klar for markedet eller som allerede kjører hos mange kunder

For at man skal være i stand til å styre kravhåndteringsprosessen blir sporbarhet for kravene en nødvendighet. Dette handler om å følge kravene fra de opprettes (opprinnelige krav) til de blir realisert i utviklingsprosjektene og tilbake igjen. Den som jobber med produktforvaltning

Opphavsrett: Knut A Strand og Stiftelsen TISIP

bør ideelt sett ha et datasystem som støtter kravhåndteringsprosessen eller minimum et sett av rutiner og verktøy (maler etc.) som støtter dette arbeidet.

Behov og forventninger kommer i forskjellige varianter, former og fasonger til utviklingsorganisasjonen og de blir gjerne navngitt med forskjellige navn som kan være avhengig av hvor detaljert de er beskrevet. Det er derfor viktig at disse kravene håndteres (registreres, sorteres, spesifiseres, etc.) slik at de kan fungere som god input til utviklingsprosjektene.

4 scenarier i forhold til kravutvikling

I første leksjon om produktforvaltning introduserte vi fire scenarier for programvareleverandører. Dette var Motorbåt-scenariet (Powerboat), Racerbåt-scenariet (Speedboat), Isbryter-scenariet (Icebreaker) og Cruiseskip-scenariet (Cruise Ship). I resten av dette avsnittet diskuterer vi disse fire scenariene i forhold til kravutvikling

- Motorbåt -scenariet fokuserer på å definere et minimumsprodukt for første kunde. Det er typisk tett dialog mellom produktforvaltning og utviklingsavdelingen. Her er det lite kunnskap om kundenes behov og hvordan produktet skal tilfredsstille disse behovene. Produktsjefen må dermed være en kreativ utforsker som kommer fram til løsninger som er attraktive for markedet. For å redusere kompleksiteten og risikoen og for å nå ut i markedet så fort som mulig, holder man programvareproduktet på et minimumsnivå. Tanken er at et minimalistisk men brukbart produkt «Minimum Viable Product» (MVP) tilbys i markedet og at markedsførings- og salgsaktiviteter starter. I motorbåt-scenariet er målet for kravutviklingen å være innovativ og kreativ, for å utvikle et produkt som er attraktivt for markedet.
- Racerbåt-scenariet fokuserer på å utvide produktets funksjonalitet for dermed å kunne øke markedsandelene. Det foregår typisk analyser av faktisk bruk av produktet, hvordan markedssituasjonen er og hvordan konkurransesituasjonen er. Produktsjefen må nå prioritere vekst for produktet, men det er ikke trivielt å avgjøre hvordan dette skal foregå. Man har typisk jobbet mot noen utvalgte, visjonære og engasjerte kunder, men må nå forsøke å forstå kravene til andre kunder generelt. For å få nye og flere kunder må man identifisere nye krav og sørge for at disse blir implementert i produktet.
- Isbryter-scenariet fokuserer også på å definere et minimumsprodukt for første kunde. Dette inkluderer grundig analyse av kundenes forretningsområde som blir input til kravhåndteringen og hvilken funksjonalitet som skal støttes av produktet. Man benytter gjerne pilotkunder, men må påse at løsningene blir generelt anvendelig og ikke for spesifikk for bestemte kunder. Kundene er ansvarlig for forretningsprosessene som gjennomføres med bruk av programvareproduktet, samt for driften av programvareproduktet i organisasjonen. Hver utgivelse av produktet er typisk et kundeprosjekt med kundespesifikke krav som identifiseres, spesifiseres og implementeres. Det er viktig å inngå avtaler i forhold til om ny funksjonalitet skal inngå i standardproduktet eller om dette skal tilhøre kunden og forvaltes i kundens miljø.
- Cruiseskip-scenariet fokuserer på å utvide produktets funksjonalitet for å oppnå høyere markedsandeler. Siden kundene ikke ønsker å installere nye versjoner ofte, i driftsmiljøene de kontrollerer, har man få men store utgivelser, typisk en til to ganger pr. år. En utfordring innenfor dette scenariet kan være at kundene kjører på forskjellige versjoner og man må hele veien sikre at man kan oppgradere fra aktuelle tidligere versjoner til siste versjon. I cruiseskip-scenariet møter produktsjefen typisk en strøm av

Opphavsrett: Knut A Strand og Stiftelsen TISIP

feilmeldinger og forespørsler om ny og utvidet funksjonalitet som kommer fra mange kilder. Kildene kan f.eks. være kunder som gjør direkte henvendelser, henvendelser via supportapparatet, salgsrepresentanter som identifiserer behov, kundeprosjekter, brukerforum og utviklingsavdelingen. Det er sentralt å etablere en kravhåndteringsprosess som identifiserer og samler kravene, før de blir evaluert og eventuelt implementert og inkludert i en utgivelse.

For mer informasjon om *«Utvikling av krav til produktene»*, se læreboken [1, s. 140–169]

1.3. Produktutvikling

Produktutvikling (eng. Development) foregår som regel i en egen avdeling (utviklingsavdelingen) og de som jobber i denne avdelingen er de som er ansvarlig for alle tekniske aspekter rundt produktene, inklusive utvikling av nye produkter, samt vedlikehold (utvikling og feilretting) på eksisterende programvare. I figur 1 består utvikling av seks områder. Dette forvaltning av programvarearkitektur (Product Architecture Management), forvaltning av utviklingsmiljø (Development Environment Management), utvikling (Development Execution), UX Design (User Experience Design), detaljert kravhåndtering (Detailed Requirements Engineering) og kvalitetsledelse (Quality Management). I kommende seks avsnitt gir vi en kort introduksjon til hvert av disse seks områdene. Det anbefales at dette leses sammen med starten av kapittel 6 i læreboken [1, s. 243–256]

1.3.1. Forvaltning av programvarearkitektur

Forvaltning av programvarearkitektur (Product Architecture Management) handler om utforming og administrasjon av programvarearkitekturen. Denne arkitekturen er den grunnleggende strukturen som bestemmer hvordan programvaren bygges, hvordan den fungerer og hvordan den kan utvides over tid.

Det er sentralt å utvikle en klar strategi for arkitekturen som definerer hvordan programvareproduktene skal være strukturert for å oppnå ønskede funksjoner og kvalitetsegenskaper. Dette inkluderer valg av arkitekturmodeller, -mønstre og -teknologier. Det er viktig å foreta en evaluering av ulike arkitekturalternativer og hvilke konsekvenser de forskjellige valgene har, før man tar en endelig beslutning om hvilken arkitektur produktet skal baseres på.

Vurdering og planlegging av hvordan produktet skal skaleres for å møte økende brukerbehov uten å miste ytelse eller stabilitet er en sentral del av dette arbeidet. Dette inkluderer tema som kapasitetsplanlegging og optimalisering av ytelse. Integrering av sikkerhets- og pålitelighetsaspekter i arkitekturen for å beskytte produktet mot trusler og sikre at det er stabilt og tilgjengelig er også sentralt. Videre kan det være hensiktsmessig å designe en modulbasert arkitektur som tillater enkelt vedlikehold, utvidelse og gjenbruk av komponenter og moduler i programvaren. Det er viktig å velge teknologier og verktøy som støtter den valgte arkitekturen og som oppfyller produktenes behov. Planlegging av hvordan programvaren skal samhandle med eksterne systemer, tjenester og komponenter gjennom definerte grensesnitt og protokoller, er også en sentral del av arkitekturarbeidet.

Opphavsrett: Knut A Strand og Stiftelsen TISIP

Det er viktig å sikre at arkitekturen er i tråd med produktets langsiktige strategi og utvikling, slik at arkitekturen støtter produktets fremtidige vekst og de endringer som kommer. Arkitekturen vil naturlig endre seg over tid og det er sentralt at disse endringene forvaltes på en god måte. Slike endringer kan være oppdateringer, forbedringer og tilpasninger i arkitekturen som gjøres for å ta høyde for endringer i krav eller teknologiske trender. En sentral del av arbeidet med systemarkitektur er utarbeidelse og vedlikehold av en fullstendig dokumentasjon, som beskriver arkitekturen på en forståelig måte for utviklingsteamet og andre som skal benytte seg av den.

Forvaltning av programvarearkitektur er viktig for å sikre at programvareprodukter er godt strukturert, skalerbare, sikre og i stand til å møte kundenes behov over tid. Det krever gjerne et tett samarbeid mellom programvarearkitekter, produktsjefen og utviklingsteamet, dersom man skal lykkes med å realisere en hensiktsmessig arkitektur for programvareproduktene. Se læreboken side 253-254 for mer informasjon om forvaltning av programvarearkitektur.

1.3.2. Forvaltning av utviklingsmiljø

Forvaltning av utviklingsmiljø (Development Environment Management) handler om å administrere miljøene som brukes i forbindelse med programvareutviklingsprosessen. Et utviklingsmiljø inkluderer maskinvare, programvare, verktøy og prosedyrer som utviklere bruker for å designe, utvikle, teste og feilsøke programvaren.

Man må beskrive utviklingsmiljøet som skal benyttes og det er derfor sentralt å utvikle en strategisk plan for hvordan utviklingsmiljøer skal konfigureres og administreres gjennom hele programvareproduktets livssyklus. Dette inkluderer valg av maskinvare, programvare og utviklingsverktøy som skal benyttes. Det er viktig å implementere systemer for versjonskontroll som lar utviklerne spore endringer i kildekoden og som gjør at de kan samarbeide effektivt. Dette inkluderer bruk av verktøy som f.eks. Git og GitHub.

Det er viktig med god administrasjon av utviklingsmiljøenes oppsett og konfigurasjon, siden dette bidrar til å sikre at miljøene er konsistente og at de oppfyller de kravene man har satt. En slik administrasjon kan gjerne automatiseres, slik at utviklingsmiljøet raskt kan generes på nye maskiner og servere. Dette kan også benyttes til å opprette kopier av utviklingsmiljøene, som gjør at utviklerne kan arbeide parallelt, samt utføre tester og simulere produksjonsmiljøer. Bruk av slike kopier av utviklingsmiljøene er spesielt viktig for å unngå konflikter og forstyrrelser i utviklingsprosessen. Man kan også automatisere rutinemessige oppgaver i utviklingsmiljøene, f.eks. med skripting. Dette for å forbedre effektiviteten og redusere faren for menneskelige feil.

Effektiv forvaltning av utviklingsmiljøene er avgjørende for å sikre at programvareutviklingen kan gjennomføres på en strukturert, effektiv og pålitelig måte. Dette bidrar til å minimere feil, redusere utviklingstiden og sikre at programvaren som produseres, er av høy kvalitet og kan levere verdi til sluttbrukerne. Utarbeidelse av nødvendig dokumentasjon for utviklingsmiljøene, inkludert installasjonsinstruksjoner, konfigurasjonsdokumentasjon og retningslinjer for bruk, er sentralt for å lykkes med dette.

Opphavsrett: Knut A Strand og Stiftelsen TISIP

1.3.3. Utvikling

Utvikling (Development Execution) omhandler alt som har med selve utviklingsarbeidet å gjøre, både innad i og på tvers av utviklingsprosjekter. Dette er stadiet der planene og strategiene som ble utarbeidet tidligere, blir satt ut i praksis, og selve programvaren blir utviklet. Ledelse av utviklingsarbeidet handler om å håndtere både kunnskap og ansatte, og å strukturere programvareutviklingsprosessene slik at utviklingsarbeidet blir mest mulig effektivitet, samtidig som kvalitetsnivået er akseptabelt. En programvareorganisasjon trenger å utvikle kompetanse og man må legge til rette for kunnskapsdeling og samarbeid. Dette må fungere på en best mulig måte selv om ressursene kommer og går. Å utvikle flere produkter (f.eks. en hel programvareportefølje) muliggjør gjenbruk av programvarekomponenter som i sin tur kan bidra til økt utviklingseffektivitet og forbedret kvalitet på programvaren. Dette må imidlertid planlegges, styres og koordineres på en god måte og dette er også en sentral del av utviklingsledelsesarbeidet.

Utviklingsarbeidet gjennomføres ofte som prosjekter og vi må planlegge og gjennomføre disse utviklingsprosjektene. Dette går ut på å definere en prosjektorganisasjon for utviklingsprosjektene, og å organisere arbeidet som denne prosjektgruppen skal gjennomføre for å ta fram programvareproduktet. Et sentralt spørsmål som man må ta stilling til når man skal gjennomføre et utviklingsprosjekt, er hvilken utviklingsmodell som skal benyttes. En utviklingsmodell kan f.eks. være basert på fossefallsmodellen (tradisjonell modell med faser som avsluttes før man går videre til neste fase), iterative modeller (hvor man gjennomfører flere iterasjoner hvor løsningene utvides og forfines gjennom hver iterasjon) eller smidige metoder (hvor man blant annet har tett dialog med kunden gjennom hele prosjektet, for å sikre at systemet faktisk implementerer det som oppdragsgiveren etterspør). Sett fra en produktsjef sitt ståsted så må man i større grad utarbeide krav i forkant dersom utviklingen er basert på fossefallsmetoden, mens man i større grad må involveres underveis dersom iterative eller smidige metoder benyttes. I dag er det mer og mer vanlig å bruke smidige tilnærminger siden dette bidrar til håndtere usikkerheten som ligger i produktutvikling av nye prosjekter på en bedre måte.
Selve kodingen handler om å skrive programvarekoden i henhold til de spesifikasjonene og kravene som er definert i tidligere faser. Dette inkluderer typisk frontend- og backend-utvikling, utforming av databasen, utvikling av brukergrensesnitt med mere. Videre er det viktig å gjennomføre omfattende testing, som sikrer at programvaren fungerer som forventet og er fri for feil. Dette inkluderer typisk enhetstesting, integrasjonstesting, systemtesting og akseptansetesting. Implementering av prosedyrer for å sikre at programvaren oppfyller de definerte kvalitetsstandardene er også sentralt. Dette kan inkludere kodeinspeksjoner og kvalitetskontroll av kode som gjennomføres av fagfeller.
Det er også viktig å opprette god teknisk dokumentasjon som forklarer programvarens arkitektur, og hvordan kildekoden fungerer. I forbindelse med dokumentasjonsarbeidet er det også sentralt å lage brukerhåndbøker og andre relevante materialer som dokumenterer systemet og dets tiltenkte bruk.

Selve utviklingsprosessen er en kritisk fase i produktforvaltningen, der produktet faktisk tar form. Effektiv gjennomføring krever god planlegging, tett samarbeid mellom utviklingsteam og produktledelse, og et fokus på kvalitet og kundeverdi.

Opphavsrett: Knut A Strand og Stiftelsen TISIP

1.3.4. UX Design

User Experience Design (UX Design) handler om å skape produkter og tjenester som gir brukerne en positiv opplevelse og som tilfredsstiller deres behov og forventninger.

Et utgangspunkt for et godt UX Design er å forstå brukerne eller målgruppens behov og preferanser. Dette kan man skaffe seg innsikt i ved f.eks. å intervjue potensielle brukere, observere brukerne når de interagerer med systemet eller tilsvarende systemer og analysere deres brukeradferd. Det kan være nyttig å utvikle såkalte wireframes eller prototyper som bidrar til å visualisere hvordan et produkt som er under utvikling vil se ut og fungere. Dette kan hjelpe oss med å teste og validere designkonsepter tidlig i utviklingsprosessen og vi kan gjerne involvere brukerne i dette testarbeidet. Det er også vanlig å opprette bruker-personas, som egentlig er fiktive representasjoner av ideelle brukere. Bruk av personas kan hjelpe oss med å forstå brukernes mål og behov, samt hvilke utfordringer de har som skal løses med systemet.

Når man jobber med UX Design er det også viktig å fokusere på brukervennlighet. Altså implementering av prinsipper for brukervennlig design, som inkluderer enkel navigasjon og mest mulig intuitive brukergrensesnitt. Man gjennomfører gjerne brukbarhetstester for å evaluere produktets brukervennlighet og identifisere potensielle problemer. Dette kan gjøres ved å observere hvordan brukerne interagerer med produktet når de f.eks. blir utfordret til å utføre en bestemt oppgave med produktet.

Det er også viktig å skape et visuelt appellerende grensesnitt med tanke på farger, typografi, layout og grafikk, som gir en positiv brukeropplevelse. Videre er såkalt responsivt design spesielt viktig dersom produktet skal gjøre på mange forskjellige enheter. Dette er et produktdesign som tilpasser seg ulike enheter og skjermstørrelser, som f.eks. datamaskiner, nettbrett og mobiltelefoner.

Godt UX Design er avgjørende for å lage programvareprodukter som er intuitive, engasjerende og tilfredsstiller brukernes behov. Det bidrar til å øke produktets verdiskapning, redusere brukerfeil og det bidrar til å bygge en lojal kundebase for produktet. Det er viktig å samarbeide med utviklerne for å sikre at designkonseptene implementeres nøyaktig og at man dermed opprettholder en god brukeropplevelse gjennom hele produktet. Se kapittel 6.2 (side 247-256) for mer informasjon om UX Design.

1.3.5. Detaljert kravhåndtering

Detaljert kravhåndtering (Detailed Requirements Engineering) fokuserer på å samle, dokumentere og spesifisere alle detaljene som er nødvendige for å utvikle et programvareprodukt, i samsvar med kundenes behov og forventninger.

Detaljert kravhåndtering er typisk en del av prosjektlederen sitt ansvar og dette følger en prosess som ligner mye på prosessen rundt utvikling av krav til produktene (se avsnitt 1.2.5 i denne leksjonen).

Opphavsrett: Knut A Strand og Stiftelsen TISIP

Når innholdet i og kravene til en utgivelse er definert, så blir disse kravene overført til prosjektet hvor dette skal realiseres. Deretter blir disse kravene som nå er prosjektets krav, ytterligere bearbeidet, beskrevet i mer detalj, estimert og knyttet opp imot prosjektets tidsplan. Noen av kravene til prosjektet trenger ikke komme fra produktet, men de kan oppstå internt i prosjektgruppen mens prosjektet pågår. Dersom man benytter smidige metoder (f.eks. Scrum) så er det slik at en større del av behovsanalysen (utvikling av kravene) gjøres underveis i utviklingsprosjektet. Det er generelt viktig at man sørger for å ha sporing på kravene. Dette handler om å vite hvor kravene kommer fra, hvilken status de har, når de har blitt (eller skal bli) realisert, hvor de har blitt (eller skal bli) realisert, etc.

Dersom vi skal se på produktsjefsrollen i forhold til utvikling av krav for prosjektet, så er det slik at produktsjefen spiller en viktig rolle i forhold til å kommunisere kravene ovenfor utviklingsprosjektet. Videre føler produktsjefen typisk ansvar i forhold til å kontrollere om framdriften i utviklingsprosjektene er tilfredsstillende og om løsningene som utvikles faktisk kan godkjennes.

Oppsummert kan vi si at detaljert kravhåndtering er en viktig fase, siden det er dette som danner grunnlaget for programvareutviklingsprosessen og hva som skal utvikles. Tydelige, detaljerte og klart forståtte krav er avgjørende for å unngå misforståelser, redusere risiko og sikre at det endelige programvareproduktet blir utviklet i samsvar med kundenes forventninger.

1.3.6. Kvalitetsledelse

Kvalitetsledelse (eng. Quality Management) kan sies å være den praksisen som en utviklingsorganisasjon implementerer for å oppnå sine kritiske suksessfaktorer og å redusere sannsynligheten for at problemer skal oppstå.

Kvalitet på programvareproduktene oppnås gjennom at man måler kvaliteten og gjennom at man etablerer rutiner for kvalitetssikring med tilhørende ansvarsfordeling. Det finnens mange organisasjoner som implementerer prosessforbedringsprogrammer for å forbedre arbeidet med forvaltning av programvarebaserte produkter og tjenester. Dette kan typisk inkludere det å legge til rette for godt test-arbeid eller god teknisk support rundt produktene. En metode eller prosess som typisk kan benyttes til å få fokus på kvaliteten i en utviklerorganisasjon er Capability Maturity Model Integration (CMMI) [8]. Dette er en metode som måler dyktigheten og modenheten til en organisasjon og plasserer organisasjonen på et nivå (1-5) avhengig av kvaliteten på arbeidet som gjøres. CMMI bidrar dermed til å avdekke hvordan prosessene i en organisasjon faktisk fungerer, samtidig som man bidrar med forslag til hvordan prosessarbeidet kan forbedres.

Produktsjefen sin oppgave i forhold til kvalitetsledelse går blant annet ut på å akseptere planleggingen og gjennomføringen av utviklingsprosjektene og de leveranser som produseres. Man må enes om omfanget i prosjektleveransene, estimatene og tidsplanene. Videre må man følge med framdriften i prosjektet og påse at prioriteringene er i overensstemmelse med hva man har blitt enig om, samt bidra inn i de forhandlinger og justeringer av planene som kommer underveis i prosjektet. Sist men ikke minst så må produktsjefen bidra til å gjennomføre tester som danner grunnlaget for å akseptere leveransene.

Opphavsrett: Knut A Strand og Stiftelsen TISIP

Fokus på kvalitetsledelse er sentralt for et selskap som ønsker å levere pålitelige og konkurransedyktige programvareprodukter som oppfyller kundenes forventninger. God kvalitetsledelse vil også bidra til å minimere risiko, redusere kostnader knyttet til feilretting og bygge et godt omdømme for både produktene og selskapet som leverer disse.

1.1. Referanseliste

[1] H.-B. Kittlaus, *Software Product Management - The ISPMA-Compliant Study Guide and Handbook, 2nd Edition*. Springer, 2022.

[2] H.-B. Kittlaus og P. N. Clough, *Software Product Management and Pricing: Key Success Factors for Software Organizations*. Springer, 2008.

[3] S. A. Fricker, «Software Product Management», i *Software for People*, Springer, 2012, s. 53–81. Åpnet: 14. mai 2025. [Online]. Tilgjengelig på: http://link.springer.com/10.1007/978-3-642-31371-4_4

[4] W. Bekkers, I. van de Weerd, M. Spruit, og S. Brinkkemper, «A framework for process improvement in software product management», i *Systems, software and services process improvement*, Springer, 2010, s. 1–12. Åpnet: 13. mai 2025. [Online]. Tilgjengelig på: http://link.springer.com/chapter/10.1007/978-3-642-15666-3_1

[5] C. Ebert og S. Brinkkemper, «Software Product Management–An Industry Evaluation», *J. Syst. Softw.*, 2014, Åpnet: 14. mai 2025. [Online]. Tilgjengelig på: http://www.sciencedirect.com/science/article/pii/S0164121214000156

[6] J. Vähäniitty, C. Lassenius, og K. Rautiainen, «An approach to product roadmapping in small software product businesses», i *ECSQ2002), Conference Notes*, Citeseer, 2002, s. 12–13.

[7] C. Ebert, «The impacts of software product management», *J. Syst. Softw.*, bd. 80, nr. 6, s. 850–861, 2007.

[8] CMMI, «Hentet 14. mai 2025 fra følgende Wikipedia-lenke: http://en.wikipedia.org/wiki/Capability_Maturity_Model_Integration», 2025.

Opphavsrett: Knut A Strand og Stiftelsen TISIP